

Закаблукова Анастасия ИВТ-1.1 2 курс

ЛАБОРАТОРНЫЙ ПРАКТИКУМ. ЧАСТЬ 5

ЛЕКЦИЯ

Флаг переноса.

Загрузка чисел 1 и FFFFh в регистры BX и AX:

```
-r bx
BX 0000
:1
-r ax
AX 0000
:ffff
-r
AX=FFFF BX=0001 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0100  NV UP EI PL NZ NA PO NC
0B11:0100 B402          MOV     AH,02
```

Запись команды “add ax, bx” в память:

```
-a 100
0B11:0100 add ax, bx
0B11:0102
-r
AX=FFFF BX=0001 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0100  NV UP EI PL NZ NA PO NC
0B11:0100 01D8          ADD     AX,BX
```

Трассировка команды:

```
-t
AX=0000 BX=0001 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NV UP EI PL ZR AC PE CY
0B11:0102 88DA          MOV     DL,BL
```

Установка IP в 100h и прибавление единицы к нулю в AX:

```
-r ip
IP 0102
:100
-t
AX=0001 BX=0001 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NV UP EI PL NZ NA PO NC
0B11:0102 88DA          MOV     DL,BL
```

Циклический сдвиг.

Запись команды и помещение в регистр BX число B7:

```
-a 100
0B11:0100 rcl bl, 1
0B11:0102
-r bx
BX 0001
:b7
-r
AX=0001 BX=00B7 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NV UP EI PL NZ NA PO NC
0B11:0102 88DA          MOV     DL,BL
-r ip
IP 0102
:100
```

```

00A7:0102 0000
-a 100
00A7:0100 mov dl, 00
00A7:0102 rcl bl, 1
00A7:0104 adc dl, 30
00A7:0107 mov ah, 02
00A7:0109 int 21
00A7:010B int 20
00A7:010D

```

Организация циклов.

Вывод четырех звездочек:

```

-a 100
0B0E:0100 mov ah, 02
0B0E:0102 mov dl, 2a
0B0E:0104 mov cx, 4
0B0E:0107 int 21
0B0E:0109 loop 107
0B0E:010B int 20
0B0E:010D
-g
****
Программа завершилась нормально

```

ЛР 4.

```

0B0E:0103 xor cx, cx
0B0E:0105 mov bx, 2
0B0E:0108 xor dx, dx
0B0E:010A div bx
0B0E:010C push dx
0B0E:010D inc cx
0B0E:010E test ax, ax
0B0E:0110 jnz 108
0B0E:0112 mov ah, 02
0B0E:0114 pop dx
0B0E:0115 add dl, 30
0B0E:0118 int 21
0B0E:011A loop 114
0B0E:011C mov dl, 20
0B0E:011E int 21
0B0E:0120 mov ax, d892
0B0E:0123 xor cx, cx
0B0E:0125 mov bx, 2
0B0E:0128 xor dx, dx
0B0E:012A div bx
0B0E:012C push dx
0B0E:012D inc cx
0B0E:012E test ax, ax
0B0E:0130 jnz 128
0B0E:0132 mov ah, 02
0B0E:0134 pop dx
0B0E:0135 add dl, 30
0B0E:0138 int 21
0B0E:013A loop 134
0B0E:013C mov dl, 30
0B0E:013E int 20
0B0E:0140
-g
01101101110100 11001100010010010

```

ЛАБОРАТОРНЫЙ ПРАКТИКУМ. ЧАСТЬ 6

ЛЕКЦИЯ

Флаги состояния.

Отрицательный флаг знака:

```

-a 100
0B0E:0100 sub ax, bx
0B0E:0102
-t
AX=0000 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0102  NV UP EI PL ZR NA PE NC
0B0E:0102 B22A          MOV     DL,2A

```

Положительный знак состояния:

```

-r ax
AX 0000
:1
-r ip
IP 0102
:100
-t
AX=0001 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0102  NU UP EI PL NZ NA PO NC
0B0E:0102 B22A          MOV     DL,2A

```

Загрузка в регистры BX и AX одинакового числа F5 и сравнение их:

```

-r ax
AX 0001
:f5
-r bx
BX 0000
:f5
-a 100
0B0E:0100 cmp ax, bx
0B0E:0102
-r ip
IP 0102
:100
-t
AX=00F5 BX=00F5 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0102  NU UP EI PL ZR NA PE NC
0B0E:0102 B22A          MOV     DL,2A

```

Команды условного перехода.

```

-a 100
0B0E:0100 sub al, 01
0B0E:0102 jnz 100
0B0E:0104 int 20
0B0E:0106
-g

```

Вывод на экран одной шестнадцатеричной цифры.

```

-r bx
BX 00F5
:f
-a 100
0B0E:0100 mov dl, b1
0B0E:0102 cmp dl, 09
0B0E:0105 ja 10c
0B0E:0107 add dl, 30
0B0E:010A jmp 10f
0B0E:010C add dl, 37
0B0E:010F mov ah, 02
0B0E:0111 int 21
0B0E:0113 in 20
0B0E:0113 int 20 ^ ошибка
0B0E:0115
-g
F
Программа завершилась нормально

```

Вывод старшей цифры двузначного шестнадцатеричного числа.

Загрузка числа 4 в CL и 5D в DL

```

-a 100
0B0E:0100 shr dl,cl
0B0E:0102
-t
AX=0100 BX=000F CX=0004 DX=0005 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B0E ES=0B0E SS=0B0E CS=0B0E IP=0102  NV UP EI PL NZ NA PE CY
0B0E:0102 80FA09          CMP     DL,09

```

ЛР 5.

```

-a 100
0B11:0100 mov ax, 5b74
0B11:0103 mov bx, d892
0B11:0106
-t
AX=5B74 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0103  NV UP EI PL NZ NA PO NC
0B11:0103 BB92D8          MOV     BX,D892
-

```

ЛАБОРАТОРНЫЙ ПРАКТИКУМ. ЧАСТЬ 7

ЛЕКЦИЯ

```

-a 100
0B0E:0100 mov dl, 41
0B0E:0102 mov cx, 000a
0B0E:0105 call 0200
0B0E:0108 loop 105
0B0E:010A int 20
0B0E:010C
-a 200
0B0E:0200 mov ah, 02
0B0E:0202 int 21
0B0E:0204 inc dl
0B0E:0206 ret
0B0E:0207
-g
*
Л 84
Программа завершилась нормально

```

```

-r ip
IP 0102
:100
-a 100
0B0E:0100 call 200
0B0E:0103 int 20
0B0E:0105
-a 200
0B0E:0200 push cx
0B0E:0201 push dx
0B0E:0202 mov dl, 0a
0B0E:0204 call 300
0B0E:0207 inc dl
0B0E:0209 pop dx
0B0E:020A pop cx
0B0E:020B ret
0B0E:020C

```

ЛАБОРАТОРНЫЙ ПРАКТИКУМ. ЧАСТЬ 8

ЛЕКЦИЯ

Ввод одной шестнадцатеричной цифры.

```

-a 100
0B11:0100 mov ah, 1
0B11:0102
-t
AX=0100 BX=0000 CX=0000 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=0B11 ES=0B11 SS=0B11 CS=0B11 IP=0102  NV UP EI PL NZ NA PO NC
0B11:0102 D583          AAD      83
-r ip
IP 0102
:100
-a 100
0B11:0100 int 21
0B11:0102
-g
3
Программа завершилась нормально

```

```

0B11:0206 mov dl, al
0B11:0208 cmp al, 30
0B11:020A jb 222
0B11:020C cmp al, 39
0B11:020E ja 215
0B11:0210 sub al, 30
0B11:0212 clc
0B11:0213 jmp 223
0B11:0215 cmp al, 41
0B11:0217 jb 222
0B11:0219 cmp al, 46
0B11:021B ja 222
0B11:021D sub al, 37
0B11:021F clc
0B11:0220 jmp 223
0B11:0222 stc
0B11:0223 jc 204
0B11:0225 mov bl, al
0B11:0227 mov ah, 2
0B11:0229 int 21
0B11:022B pop dx
0B11:022C pop ax
0B11:022D ret
0B11:022E

```

```

-a 100
0B11:0100 call 200
0B11:0103 int 20
0B11:0105
-g
F
Программа завершилась нормально

```

ЛР 6

```

-a 100
0B11:0100 mov cx, 4
0B11:0103 mov bx, 0
0B11:0106 lodsb
0B11:0107 cmp al, 30
0B11:0109 jl 190
0B11:0109 jg 190 ^ osh
0B11:0109 cmp al, 39
0B11:010B jg 170 ^ osh
0B11:010D sub al, 30
0B11:010F jmp 180
0B11:0111 cmp al, 46
0B11:0113 jg 190
0B11:0115 sub al, 41
0B11:0117 add al, 0a
0B11:0119 shl bx, 4
0B11:0119 add bx, ax
0B11:011B loop 156
0B11:011D ret
0B11:011E mov ah, 4c
0B11:0120 int 21

```

```

-a 200
0B11:0200 mov cx, 10
0B11:0203 mov dx, [si]
0B11:0205 shl dx, 1
0B11:0207 jc 220
0B11:0209 mov dl, 30
0B11:020B jmp 230
0B11:020D mov dl, 31
0B11:020F mov ah, 2
0B11:0211 int 21
0B11:0213 loop 203
0B11:0215 ret
0B11:0216

```

Адрес е 120

0120: 56 76 65 64 69 74 65 20 73 6F 64 65 72 6A 69 6D

012E: 6F 65 20 72 65 67 69 73 74 72 61 20 41 58 3A 24

```

-e 138
0B11:0138 CD.28 21.41 72.58 39.29 8B.20 FA.3d 33.20 C0.24

```

```

-e 148
0B11:0148 C0.56 74.76 0C.65 32.64 E4.69 E8.74 23.65 E2.20
0B11:0150 74.73 F1.6f 47.64 FE.65 C4.72 EB.6a EC.69

```

```

-g
Uvedite soderjimo regis<AX> = F46B
Uvedite sodejimo regis<BX> = 5A0C
AX=1111010001101011
BX=0101101000001100

```

ЛАБОРАТОРНЫЙ ПРАКТИКУМ. ЧАСТЬ 9

ЛР 7

Часть 1

1.Создание каталога PROBA в корневом каталоге логического диска C:

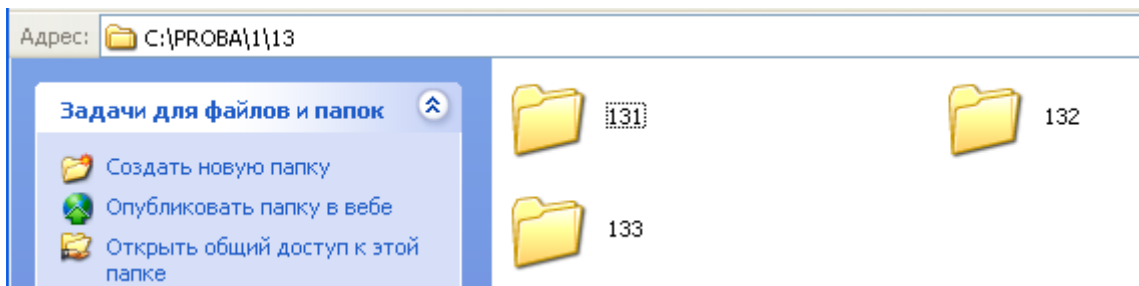
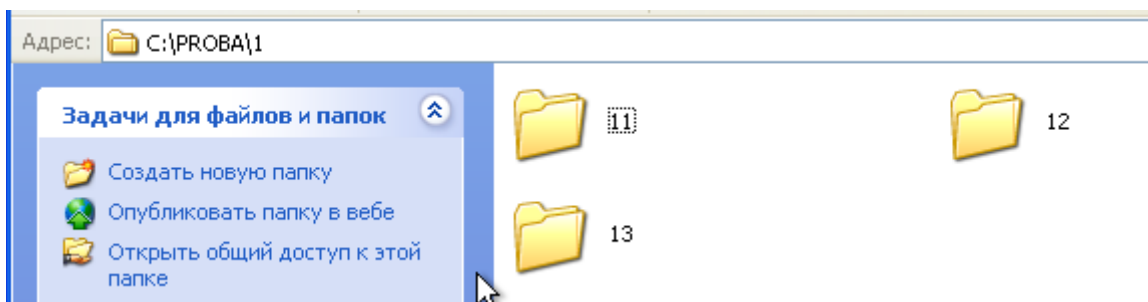
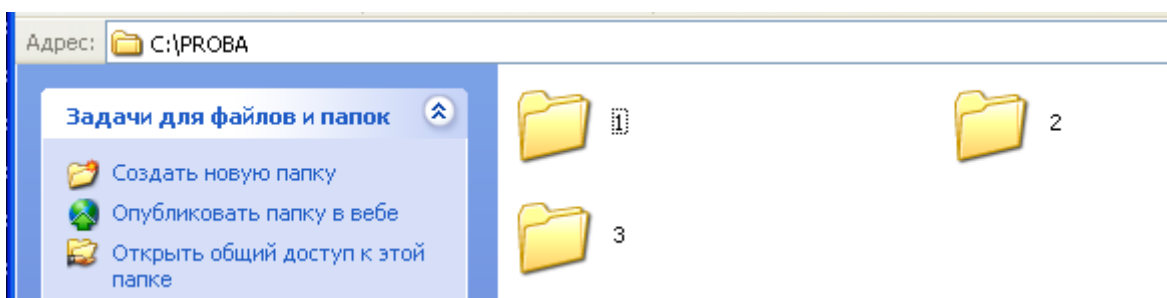
```

C:\DOCUME~1\9335~1>cd c:\
C:\>md PROBA
C:\>cd PROBA
C:\PROBA>

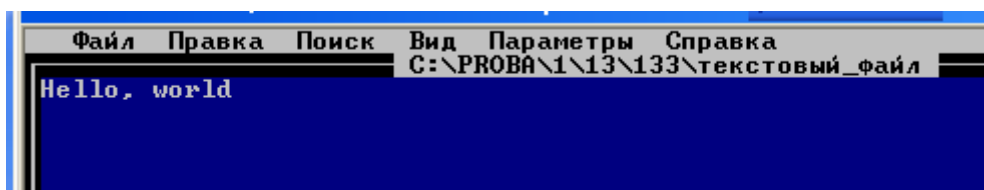
```

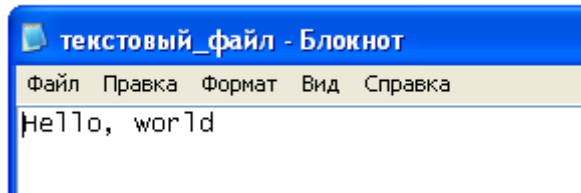
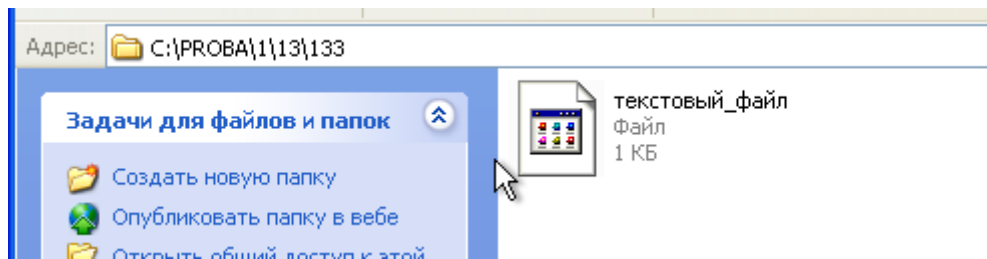
2.Создать дерево каталогов с корнем PROBA

```
C:\PROBA>md 1
C:\PROBA>md 2
C:\PROBA>md 3
C:\PROBA>cd 1
C:\PROBA\1>md 11
C:\PROBA\1>md 12
C:\PROBA\1>md 13
C:\PROBA\1>cd 13
C:\PROBA\1\13>md 131
C:\PROBA\1\13>md 132
C:\PROBA\1\13>md 133
```



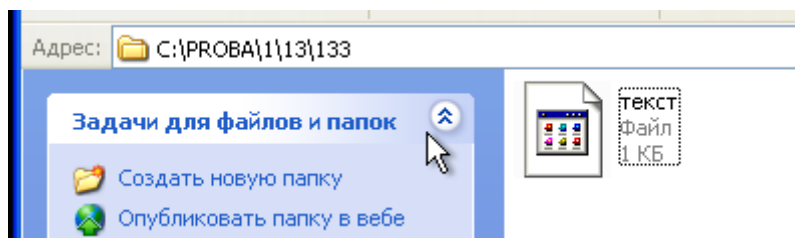
3. В каталоге 133 создать текстовый файл, содержащий несколько слов русского или английского языка.





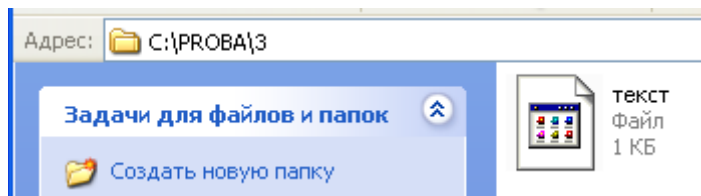
4.Переименовать файл

```
C:\PROBA\1\13\133>ren текстовый_файл текст
```



5.Одной командой выполнить копирование в каталог 3.

```
C:\PROBA\1\13\133>сору текст с:\PROBA\3
Скопировано файлов: 1.
```

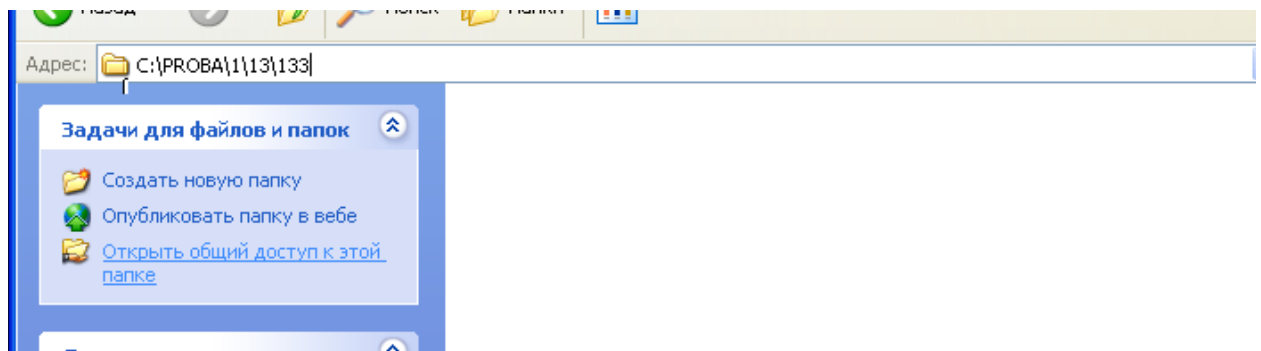
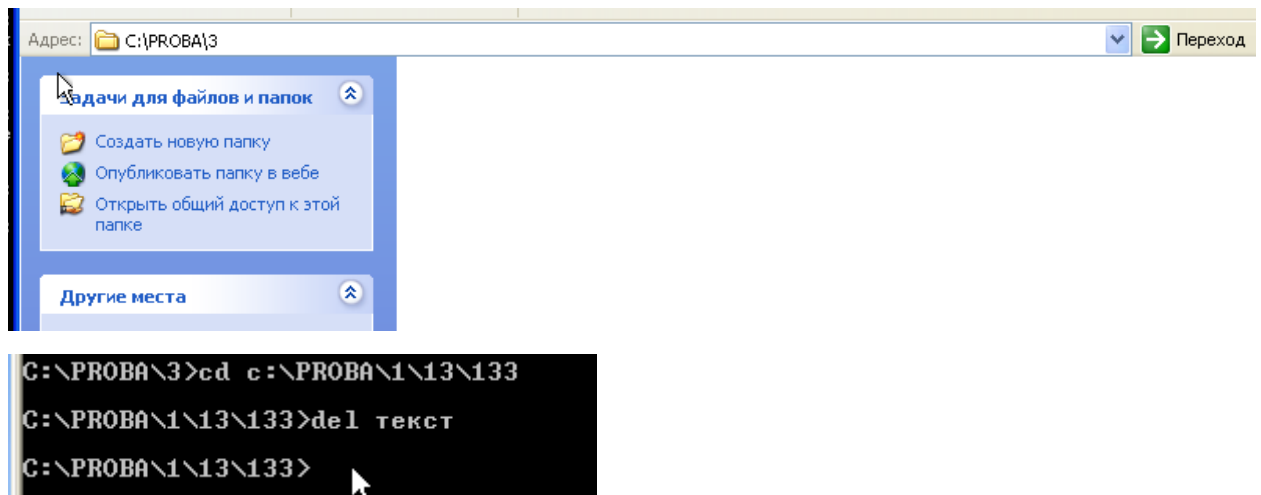


6.Вывести на экран содержимое текстового файла.

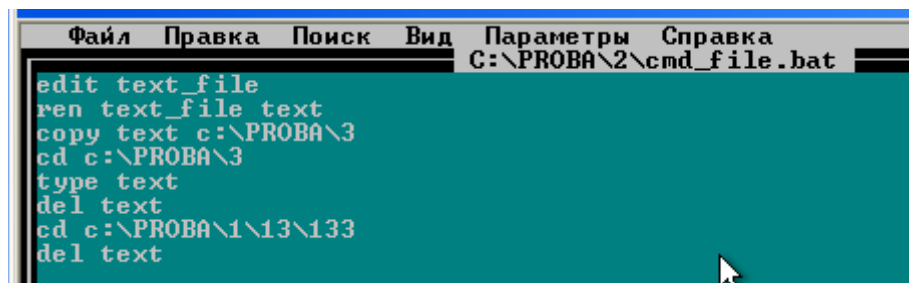
```
C:\PROBA\1\13\133>cd c:\PROBA\3
C:\PROBA\3>type текст
Hello, world
```

7.Выполнить уничтожение обоих текстовых файлов.

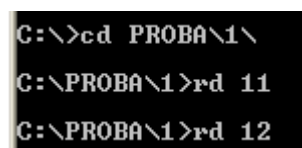
```
C:\PROBA\3>del текст
```

8. Поместить в каталог 2, а затем выполнить командный файл, содержащий команды, перечисленные в предыдущих шагах.



9. Выполнить уничтожение всех созданных файлов и каталогов.



```

C:\PROBA\1\13>rd 131
C:\PROBA\1\13>rd 132
C:\PROBA\1\13>rd 133
C:\PROBA\1\13>cd ..
C:\PROBA\1>rd 13
C:\PROBA\1>cd ..
C:\PROBA>rd 1
C:\PROBA>rd 2
C:\PROBA>rd 3
C:\PROBA>cd ..
C:\>rd PROBA
C:\>

```

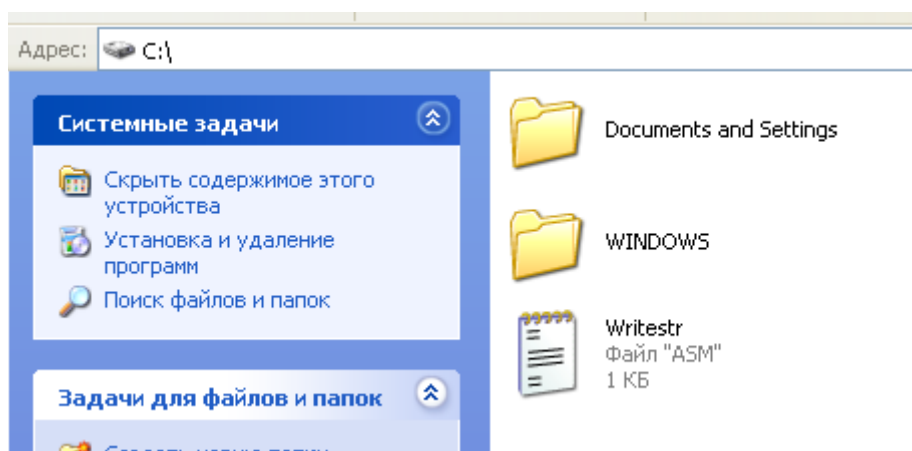
Часть 2.

С темой была ознакомлена в курсе «Операционные системы». Информация понята и усвоена.

ЛАБОРАТОРНЫЙ ПРАКТИКУМ. ЧАСТЬ 10

ЛЕКЦИЯ

Создать файл Writestr.asm и поместить в него программу:



Вывести содержимое программы:

```

C:\>type Writestr.asm
org 100h
mov ah, 2h
mov dl, 2ah
int 21h
int 20h

```

```
section .data
    hex_digit db '00$', 0
```

```
section .text
    global _start
```

```
_start:
    mov eax, 4
    mov ebx, 1
    mov ecx, hex_digit
    mov edx, 3
    int 0x80

    mov eax, 1
    xor ebx, ebx
    int 0x80
```

Compilation time: 0.14 sec, absolute running time: 0.16 sec, cpu time: 0.01 sec, memory peak: 6 Mb, absolute service time: 0.5 sec
00\$

ЛАБОРАТОРНЫЙ ПРАКТИКУМ. ЧАСТЬ 11

ЛР 9

main.asm

```
section .data
    word_data dw 3039h
section .bss
section .text
    global _start
_start:
    mov dx, word_data
    call Test_write_word_dec
    call Test_write_word_hex
    mov eax, 60
    xor edi, edisyscall
Test_write_word_dec:
    mov dx, word_data
    call Write_word_hex
    ret
Test_write_word_hex:
    mov dx, word_data
    call Write_word_hex
    ret

#include "write_digit_hex.asm"
#include "write_char.asm"
#include "write_word_dec.asm"
#include "write_word_hex.asm"
#include "write_byte_hex.asm"
```

write_word_dec.asm

```

section .text

Write_word_dec:
    push cx
    push ax

    mov cx, 0
    mov ax, dx
    mov bx, 10

.divide_loop:
    xor dx, dx
    div bx
    push dx
    inc cx
    test ax, ax
    jnz .divide_loop

.print_loop:
    pop dx
    add dl, '0'
    call Write_char
    loop .print_loop

    pop ax
    pop cx
    ret

```

write_word_hex.asm

```

section .text

Write_word_hex:
    push ax
    push dx

    mov al, dh
    call Write_byte_hex

    mov al, dl
    call Write_byte_hex

    pop dx
    pop ax
    ret

```

write_byte_hex.asm

```

section .text

write_byte_hex:
    push bx
    mov bx, 16

    mov ah, 0
    mov al, dlshr al, 4
    call write_digit_hex

    mov al, dl
    and al, 0Fh
    call write_digit_hex

    pop bx
    ret

```

write_digit_hex.asm

```

section .text

write_digit_hex:
    cmp al, 10
    jb .digit
    add al, 'A' - 10
    jmp .done

.digit:
    add al, '0'

.done:
    call write_char
    ret

```

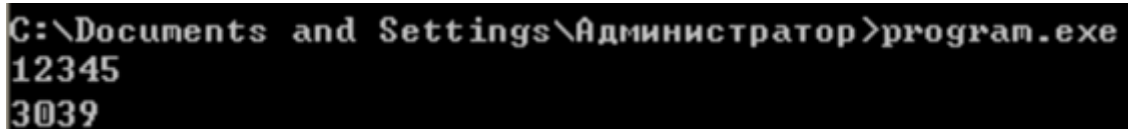
write_char.asm

```

section .text

write_char:
    mov eax, 1
    mov edi, 1
    mov rsi, rsp
    mov edx, 1
    syscall
    ret

```



```

C:\Documents and Settings\Администратор>program.exe
12345
3039

```

ЛАБОРАТОРНЫЙ ПРАКТИКУМ. ЧАСТЬ 12

ЛР 10

source.asm

```
%include "write_word.asm"
%include "write_byte.asm"
%include "write_char.asm"
%include "write_address.asm"
%include "write_digit.asm"
```

```
section .data
    word_data dw 3039h
```

```
section .bss
```

```
section .text
    global _start
```

```
_start:
    mov rdx, word_data
    call Test_write_word_dec
    movzx rdx, word_data
    call Test_write_word_hex

    mov eax, 60
    xor edi, edi
    syscall
```

```
Test_write_word_dec:
    movzx rdx, word_data
    call Write_word_dec
    ret
```

```
Test_write_word_hex:
    movzx rdx, word_data
    call Write_word_hex
    ret
```

write_word.asm

```
%include "write_byte.asm"
%include "write_char.asm"
%include "write_address.asm"

Write_word_dec:
    push rcx
    push rax

    mov rcx, 0
    mov rax, rdx
    mov rbx, 10

.divide_loop:
    xcr rdx, rdx
    div rbx
    push rdx
    inc rcx
    test rax, rax
    jnr .divide_loop

.print_loop:
    pop rdx
    add dl, '0'
    call Write_char
    loop .print_loop

    pop rax
    pop rcx
    ret

Write_word_hex:
    push rax
    push rdx

    movzx rax
    call Write_byte_hex
    movzx rax, dl
    call Write_byte_hex

    movzx rax, dh
    call Write_char_reprexentation
    movzx rax, dl
    call Write_char_reprexentation

    pop rdx
    pop rax
    ret
```

write_byte.asm

```
%include "write_digit.asm"
%include "write_char.asm"
```

```
Write_byte_hex:
    push rbx
    mov rbx, 16

    mov al, dl
    shr al, 4
    call Write_digit_hex

    mov al, dl and al, 0Fh
    call Write_digit_hex

    pop rbx
    ret
```

write_char.asm

```
%include "write_byte.asm"

Write_char:
    cmp al, 20h
    je .check_special_characters

    mov eax, 1
    mov edi, 1
    mov rsi, rsp
    mov edx, 1
```

```
C:\Documents and Settings\Администратор>program.exe
3039 41 42 43 44 45 46 47 00 20 1F 80 81 82 83 84 85 ABCDEFG. . 47 47
```